DSIC - Sprint 0 - 2

Sistema de monitorización ambiental mediante BLE

1. Objetivo y alcance

El sistema propuesto recibe tramas BLE de un nodo sensor, las recoge mediante una aplicación Android, las envía por HTTPS a un servidor REST, aplica reglas de negocio, persiste las mediciones en una base de datos y permite consultarlas desde un navegador.

Arquitectura objetivo

Nodo sensor XBW -> BLE -> Aplicación Android -> HTTPS/JSON -> Servidor REST -> Lógica del negocio -> Base de datos -> Aplicación web

Supuestos

- El identificador lógico estable del nodo es nodeCode; la dirección BLE puede cambiar.
- Los valores aún no interpretados se conservan como rawData.
- La fecha de recepción se asigna en el servidor.
- La lógica fake usa datos sintéticos y no accede a Bluetooth ni a la red real.
- Las credenciales nunca se incluyen directamente en el código fuente.

Notación utilizada

Se aplica la notación de la asignatura: tipos con mayúscula inicial; variables con minúscula; funciones con verbo; parámetros nombre:Tipo; funciones Entrada -> funcion() -> Salida; errores mediante Salida | Error; tuplas Tipo = (campo:Tipo, ...); asignación variable <- valor; condiciones si / si no; protocolos mediante mensajes en mayúsculas y parámetros entre ángulos.

2. Tipos del dominio

EstadoNodo = {ACTIVO, INACTIVO, DESCONOCIDO}

Error = {
    API_KEY_INVALIDA, JSON_INVALIDO, NODO_INEXISTENTE,
    MEDICION_INVALIDA, MEDICION_DUPLICADA, ERROR_PERSISTENCIA
}

Nodo = (
    idNodo:N, nodeCode:Texto, nombre:Texto, descripcion:Texto, activo:VoF,
    fechaCreacion:Texto
)

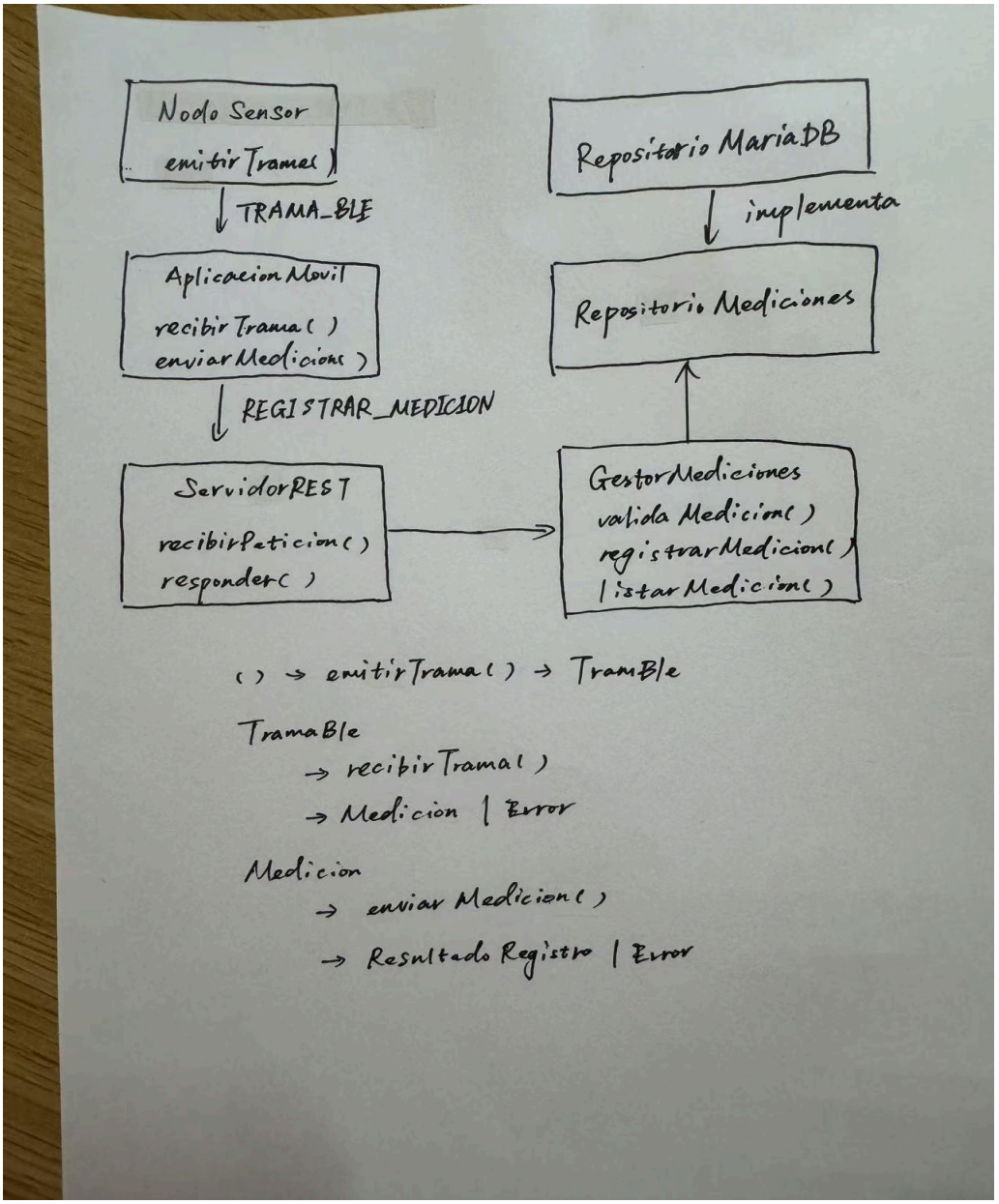
Medicion = (
    idMedicion:N, nodeCode:Texto, nombreDispositivo:Texto,
    direccionDispositivo:Texto, rssi:Z, rawData:Texto,
    temperatura:R, co2:R, ruido:R, fechaMedicion:Texto,
    fechaRecepcion:Texto
)

FiltroMediciones = (
    nodeCode:Texto, desde:Texto, hasta:Texto, limite:N, desplazamiento:N
)

ResultadoRegistro = (idMedicion:N, fechaRecepcion:Texto)

Observación: temperatura, CO2 y ruido son opcionales hasta disponer de la codificación definitiva de la trama BLE.

3. Diseño general de arquitectura

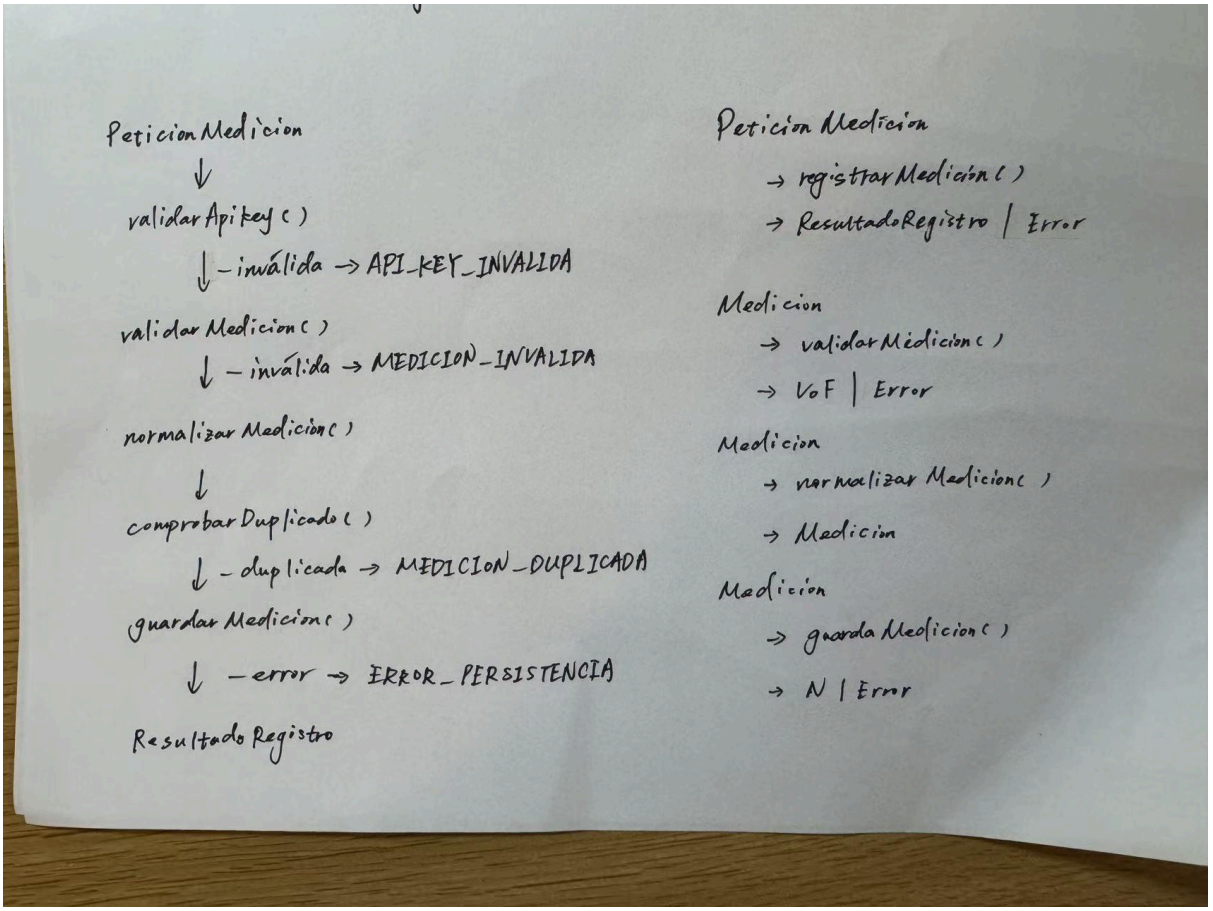


Funciones principales

() -> emitirTrama() -> TramaBle
TramaBle -> recibirTrama() -> Medicion | Error
Medicion -> enviarMedicion() -> ResultadoRegistro | Error

Medicion -> registrarMedicion() -> ResultadoRegistro | Error
FiltroMediciones -> listarMediciones() -> [Medicion] | Error

4. Diseño de la lógica del negocio



Operaciones

PeticionMedicion -> registrarMedicion() -> ResultadoRegistro | Error

Medicion -> validarMedicion() -> VoF | Error

Medicion -> normalizarMedicion() -> Medicion

Medicion -> comprobarDuplicado() -> VoF | Error

Medicion -> guardarMedicion() -> N | Error

FiltroMediciones -> listarMediciones() -> [Medicion] | Error

Texto -> obtenerUltimaMedicion() -> Medicion | Error

Reglas del negocio

· RN-01. nodeCode y nombreDispositivo son obligatorios.

· RN-02. rssi debe ser entero y técnicamente válido.

· RN-03. rawData debe ser una cadena hexadecimal válida.

· RN-04. Toda escritura requiere una API Key válida.

· RN-05. fechaRecepcion la asigna el servidor.

· RN-06. Una medición almacenada no se modifica desde el teléfono.

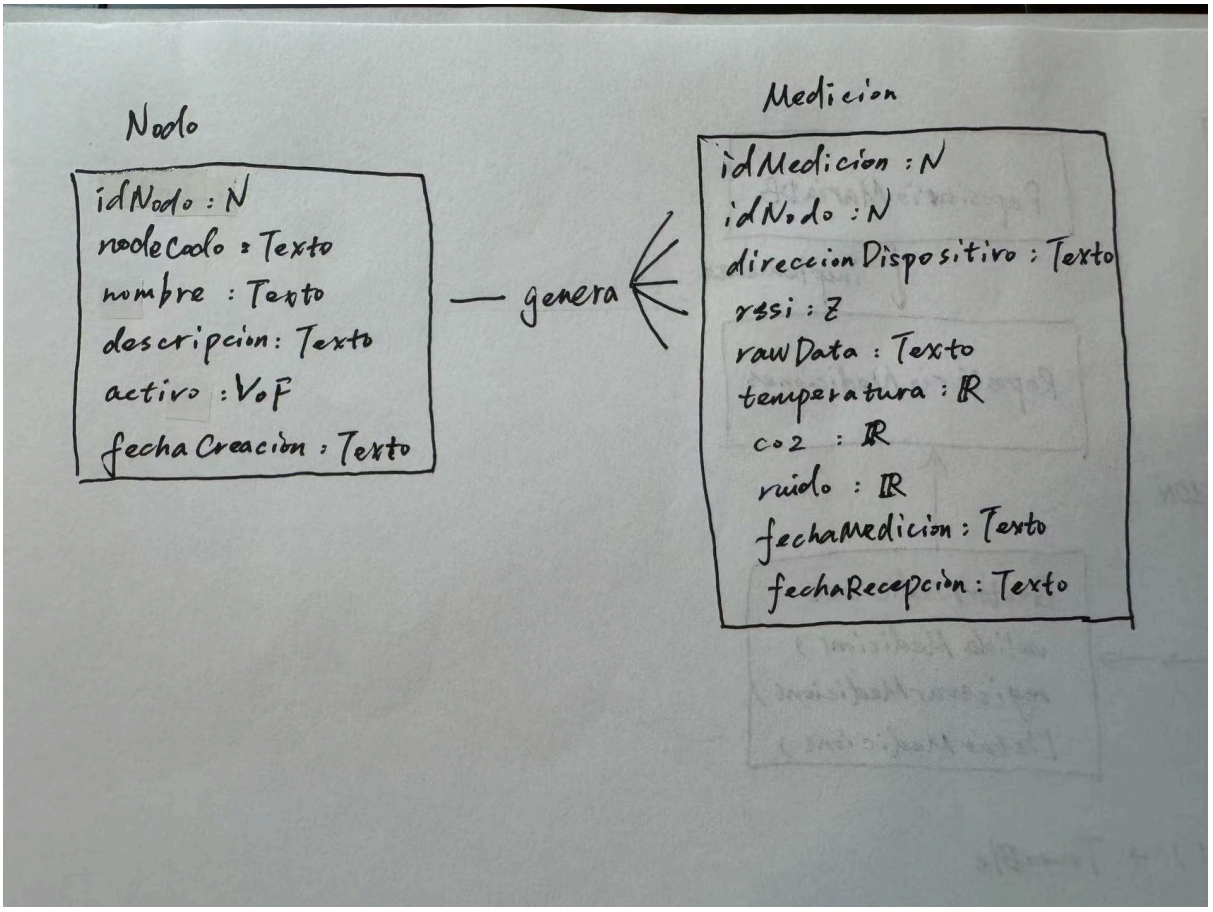
· RN-07. Se deben detectar reenvíos accidentales.

· RN-08. La dirección BLE no es un identificador permanente.

Algoritmo resumido

```
PeticionMedicion -> registrarMedicion() -> ResultadoRegistro | Error
si apiKey no es válida
        error <- API_KEY_INVALIDA
si no
        si validarMedicion(peticion) es falso
                error <- MEDICION_INVALIDA
si no
        medicion <- normalizarMedicion(peticion)
        si comprobarDuplicado(medicion)
                error <- MEDICION_DUPLICADA
        si no
                id <- guardarMedicion(medicion)
                resultado <- (idMedicion:id, fechaRecepcion:fechaServidor)
```

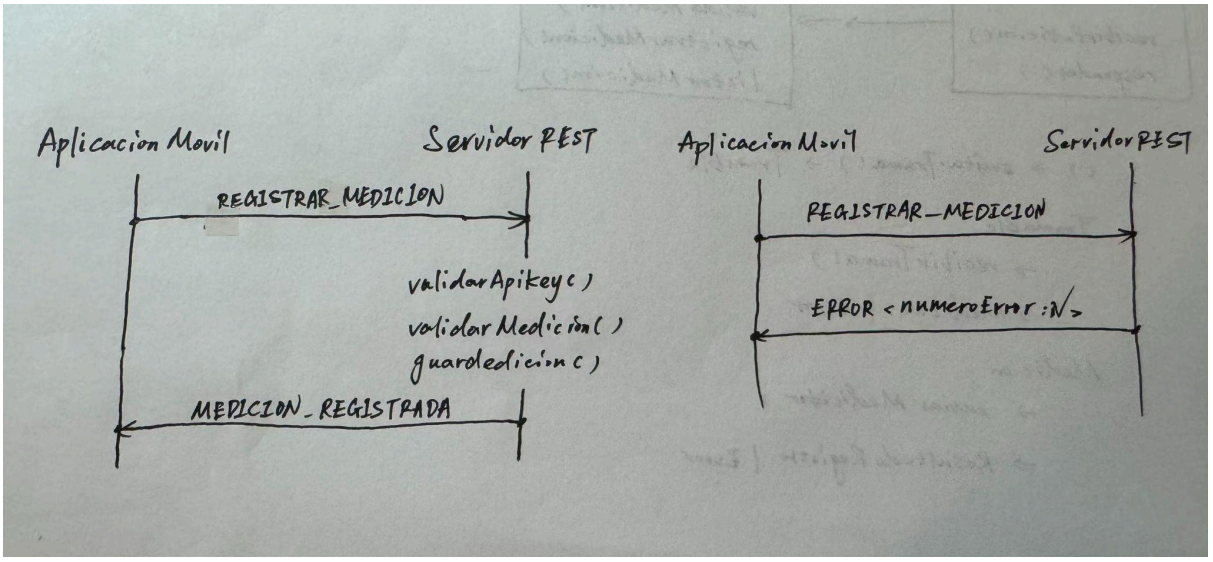
5. Diseño de la base de datos



Restricciones

- nodeCode es único y obligatorio.
- Cada Medicion pertenece exactamente a un Nodo existente.
- rawData y rssi son obligatorios.
- temperatura, co2, ruido y fechaMedicion pueden ser nulos.
- Índices: nodeCode, fechaRecepcion y (idNodo, fechaRecepcion).

6. Diseño de rutas REST y protocolo



Rutas

GET /api/health : Comprueba el estado del servicio.

POST /api/measurements : Registra una medición.

GET /api/measurements : Lista con filtros y paginación.

GET /api/measurements/{id} : Consulta una medición.

GET /api/nodes : Lista nodos.

GET /api/nodes/{nodeCode} : Consulta un nodo.

GET /api/nodes/{nodeCode}/measurements/latest : Obtiene la última medición.

Mensajes de protocolo

REGISTRAR_MEDICION

<nodeCode:Texto>

<nombreDispositivo:Texto>

<direccionDispositivo:Texto> <rssi:Z>

<rawData:Texto>

MEDICION_REGISTRADA

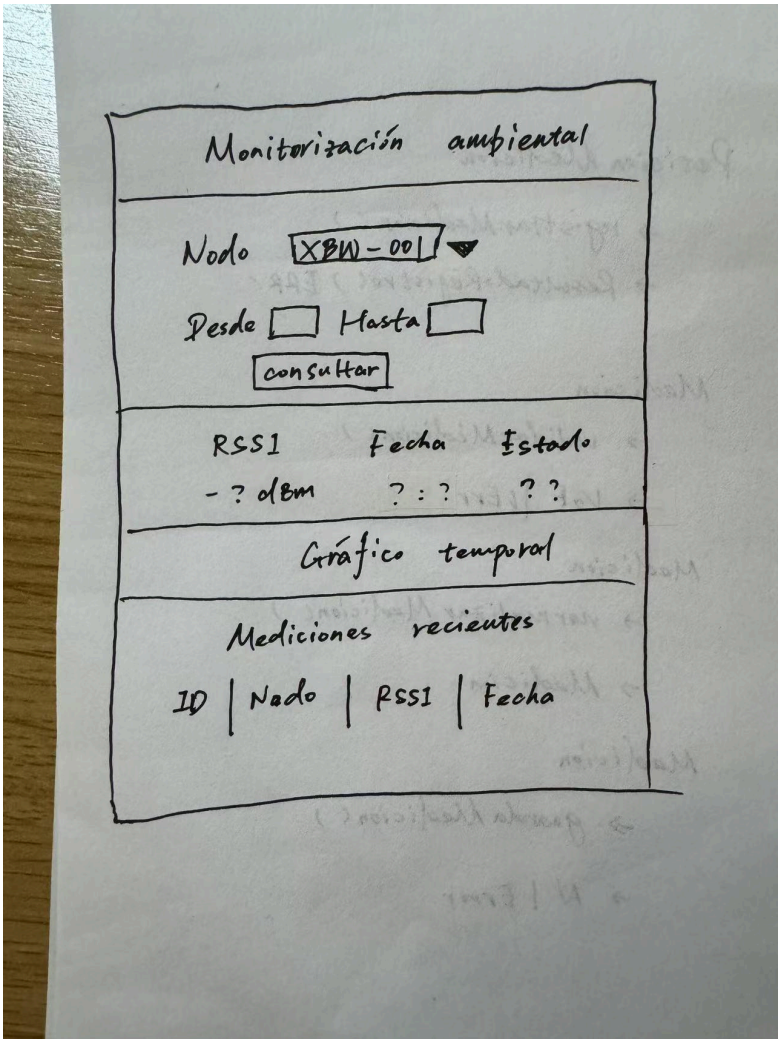
<idMedicion:N>

<fechaRecepcion:Texto>

ERROR <numeroError:N>

1 API Key inválida; 2 JSON inválido; 3 medición inválida; 4 nodo inexistente; 5 duplicado; 6 error interno.

7. Diseño de la interfaz gráfica web



Requisitos UX

- Diseño responsive para ordenador, tableta y teléfono.
- Navegación por teclado y etiquetas accesibles.
- No depender solo del color para comunicar estados.
- Mensajes claros para carga, vacío, timeout, 401 y 500.
- rawData se muestra solo en el detalle.
- El modo fake se identifica visualmente.

8. Prompt para generar la base de datos

Diseño gráfico de referencia

Usar la figura indicada en las secciones anteriores como anexo obligatorio del prompt.

Prompt

Actúa como especialista en bases de datos relacionales para sistemas IoT. Genera la definición de una base de datos MariaDB 10.11 utilizando como fuente obligatoria el diseño gráfico de la sección 5. No cambies entidades, cardinalidades ni restricciones sin justificarlo.

Crea las tablas nodes y measurements. nodes debe incluir id_node, node_code único, name, description, active y created_at. measurements debe incluir id_measurement, id_node como clave foránea, device_address, rssi, raw_data, temperature, co2, noise, measured_at y received_at. La dirección BLE no es un identificador permanente. temperature, co2, noise y measured_at pueden ser nulos; raw_data y rssi son obligatorios.

Genera: script de creación, script de eliminación para pruebas, datos iniciales para XBW-001, consultas de inserción y consulta, índices para node_code, received_at y (id_node, received_at), y una justificación breve. Incluye comentarios de cabecera con la notación oficial para cada operación.

Genera tests automáticos para inserción de nodo, inserción de medición, rechazo de nodo inexistente, orden de últimas mediciones, integridad referencial y duplicación de node_code. Marca todos los datos de prueba como sintéticos. No incluyas credenciales reales.

Criterios de aceptación

- ☐ La salida respeta el diseño y la separación por capas.
- ☐ Las funciones incluyen cabeceras en la notación oficial.
- ☐ Se incluyen tests automáticos ejecutables.
- ☐ No aparecen credenciales reales ni datos sensibles.
- ☐ Los errores se gestionan sin revelar información interna.

9. Prompt para generar la lógica real del negocio

Diseño gráfico de referencia

Usar la figura indicada en las secciones anteriores como anexo obligatorio del prompt.

Prompt

Actúa como arquitecto de software backend. Genera la lógica del negocio real siguiendo el diseño de la sección 4. Debe ser independiente de HTTP y de MariaDB: no recibe Request, no devuelve estados HTTP y accede a persistencia mediante interfaces de repositorio.

Implementa registrarNodo(), obtenerNodo(), registrarMedicion(), obtenerMedicion(), listarMediciones(), obtenerUltimaMedicion(), validarMedicion(), normalizarMedicion() y comprobarDuplicado(). Aplica todas las reglas RN-01 a RN-08. No inventes temperatura, CO2 ni ruido; conserva rawData cuando no se pueda interpretar.

Usa entidades de dominio, servicios de aplicación, interfaces de repositorio, implementaciones separadas y excepciones de negocio. Para cada función incluye una cabecera con Entrada -> funcion() -> Salida | Error, propósito, parámetros, precondiciones, postcondiciones, errores y efectos laterales.

Genera tests unitarios con repositorios fake para medición válida, nombre vacío, RSSI inválido, hexadecimal inválido, nodo inexistente, duplicado, persistencia y última medición. Los tests no dependen de red ni base de datos real.

Criterios de aceptación

- ☐ La salida respeta el diseño y la separación por capas.
- ☐ Las funciones incluyen cabeceras en la notación oficial.
- ☐ Se incluyen tests automáticos ejecutables.
- ☐ No aparecen credenciales reales ni datos sensibles.
- ☐ Los errores se gestionan sin revelar información interna.

10. Prompt para generar el servidor REST

Diseño gráfico de referencia

Usar la figura indicada en las secciones anteriores como anexo obligatorio del prompt.

Prompt

Actúa como desarrollador de APIs REST seguras. Genera el servidor REST conforme a las rutas y al protocolo de la sección 6. La capa REST solo traduce HTTP a llamadas de negocio: no implementa reglas del negocio ni SQL.

Implementa GET /api/health; POST /api/measurements; GET /api/measurements con limit, offset, node_code, from y to; GET /api/measurements/{id}; GET /api/nodes; GET /api/nodes/{nodeCode}; y GET /api/nodes/{nodeCode}/measurements/latest.

Acepta JSON; valida X-API-Key; limita el cuerpo; devuelve JSON uniforme; usa 200, 201, 400, 401, 404, 409 y 500; registra errores técnicos sin exponer SQL ni credenciales; implementa paginación y CORS solo para orígenes autorizados.

Incluye cabeceras de función con la notación oficial, comentarios breves y ejemplos sin credenciales reales. Genera tests automáticos para health, creación correcta, JSON inválido, API Key ausente/incorrecta, campos ausentes, nodo inexistente, duplicado, consulta, paginación y error interno sin filtración.

Criterios de aceptación

- ☐ La salida respeta el diseño y la separación por capas.
- ☐ Las funciones incluyen cabeceras en la notación oficial.
- ☐ Se incluyen tests automáticos ejecutables.
- ☐ No aparecen credenciales reales ni datos sensibles.
- ☐ Los errores se gestionan sin revelar información interna.

11. Prompt para generar la lógica fake del teléfono

Diseño gráfico de referencia

Usar la figura indicada en las secciones anteriores como anexo obligatorio del prompt.

Prompt

Actúa como desarrollador Android especializado en IoT. Genera una lógica fake que implemente las mismas interfaces que la futura lógica real y permita probar la aplicación sin nodo BLE ni servidor.

Simula: Bluetooth no disponible, permisos denegados, escaneo, XBW encontrado/no encontrado, medición válida, error BLE, subida en curso, subida correcta, HTTP 400/401/500, timeout y modo offline. Los datos son sintéticos, por ejemplo nodeCode XBW-001, nombre XBW, dirección FA:KE:00:00:00:01, RSSI -42 y rawData 02011A03030A180409584257.

No accedas a Bluetooth ni a la red; permite configurar retraso y resultado; no bloquees el hilo principal; separa la fake de la interfaz; permite sustituirla por la implementación real; muestra MODO FAKE; no incluyas credenciales.

Incluye cabeceras con la notación oficial y tests para estado inicial, escaneo, encontrado/no encontrado, subida correcta, timeout, 401, reintento y conservación de la última medición.

Criterios de aceptación

- ☐ La salida respeta el diseño y la separación por capas.
- ☐ Las funciones incluyen cabeceras en la notación oficial.
- ☐ Se incluyen tests automáticos ejecutables.
- ☐ No aparecen credenciales reales ni datos sensibles.
- ☐ Los errores se gestionan sin revelar información interna

12. Prompt para generar la lógica fake del navegador

Diseño gráfico de referencia

Usar la figura indicada en las secciones anteriores como anexo obligatorio del prompt.

Prompt

Actúa como desarrollador frontend. Genera la lógica fake del navegador siguiendo las rutas de la sección 6 y el wireframe de la sección 7. Debe implementar las mismas interfaces que el cliente REST real: listNodes(), getNode(), listMeasurements(), getMeasurement() y getLatestMeasurement().

Simula lista vacía, una o varias mediciones, paginación, filtros por nodo y fechas, nodo activo/inactivo, 400/401/404/500, timeout y conexión lenta. Los datos se guardan en ficheros separados, son sintéticos y tienen fechas reproducibles.

No realices peticiones reales; permite cambiar entre fake y real por configuración; muestra MODO DEMOSTRACIÓN; gestiona carga, éxito, vacío y error; no dupliques reglas del backend ni incluyas credenciales.

Incluye cabeceras según la notación oficial y tests para carga, vacío, filtros, paginación, timeout, 401, 500 y cambio fake/real.

Criterios de aceptación

- ☐ La salida respeta el diseño y la separación por capas.
- ☐ Las funciones incluyen cabeceras en la notación oficial.
- ☐ Se incluyen tests automáticos ejecutables.
- ☐ No aparecen credenciales reales ni datos sensibles.
- ☐ Los errores se gestionan sin revelar información interna.

13. Prompt para generar la UX del navegador

Diseño gráfico de referencia

Usar la figura indicada en las secciones anteriores como anexo obligatorio del prompt.

Prompt

Actúa como especialista en UX, accesibilidad y frontend. Genera la interfaz a partir del wireframe de la sección 7, respetando su jerarquía. Incluye cabecera, selector de nodo, filtros desde/hasta, botón Consultar, tarjetas de último RSSI/fecha/estado, gráfico temporal, tabla de mediciones y detalle con rawData.

Diseña los estados cargando, con datos, sin datos, validación, 401, servidor no disponible, timeout y offline. El modo fake debe ser visible.

La interfaz debe ser responsive, navegable por teclado, con etiquetas accesibles, contraste suficiente y estados que no dependan solo del color. Separa componentes visuales de acceso a datos, reutiliza componentes, evita valores fijos y no muestra credenciales ni errores internos.

Incluye cabeceras de función con la notación oficial y comentarios breves en decisiones de accesibilidad. Genera tests para renderizado, carga, vacío, error, filtros, selección de nodo, teclado, accesibilidad y tamaños móvil/escritorio.

Criterios de aceptación

- ☐ La salida respeta el diseño y la separación por capas.
- ☐ Las funciones incluyen cabeceras en la notación oficial.
- ☐ Se incluyen tests automáticos ejecutables.
- ☐ No aparecen credenciales reales ni datos sensibles.
- ☐ Los errores se gestionan sin revelar información interna.